جلسه‌ی ۷

۷

SQL کاربردی برای تحلیل داده

زبان مشترک پایگاه داده‌ها و یکی از پرتکرارترین سؤال‌های مصاحبه‌ی شغلی

ترم دوم: علم داده | کیوان محمودی

پایگاه داده‌ی رابطه‌ای

۱ جدول

داده در چند جدول مرتبط: Customer، Invoice، InvoiceLine، Track، Genre، Album، Artist

۲ کلید اصلی

ستونی که هر سطر را یکتا مشخص می‌کند؛ مثل CustomerId در Customer

۳ کلید خارجی

ستونی که به کلید اصلی جدول دیگر اشاره می‌کند؛ مثل CustomerId در Invoice

از پایتون به SQLite

- SQLite یک پایگاه داده‌ی کامل در قالب یک فایل است و در خود پایتون وجود دارد
- نتیجه‌ی هر پرسش مستقیم یک DataFrame است
- همین دستورها تقریباً بدون تغییر در PostgreSQL و MySQL کار می‌کنند

```
import sqlite3
conn = sqlite3.connect("chinook.sqlite")

def sql(query, params=None):
    return pd.read_sql_query(
        query, conn, params=params)

sql("""
SELECT FirstName, LastName, Country
FROM Customer
LIMIT 5
""")
```

ترتیب نوشتن یک پرسش



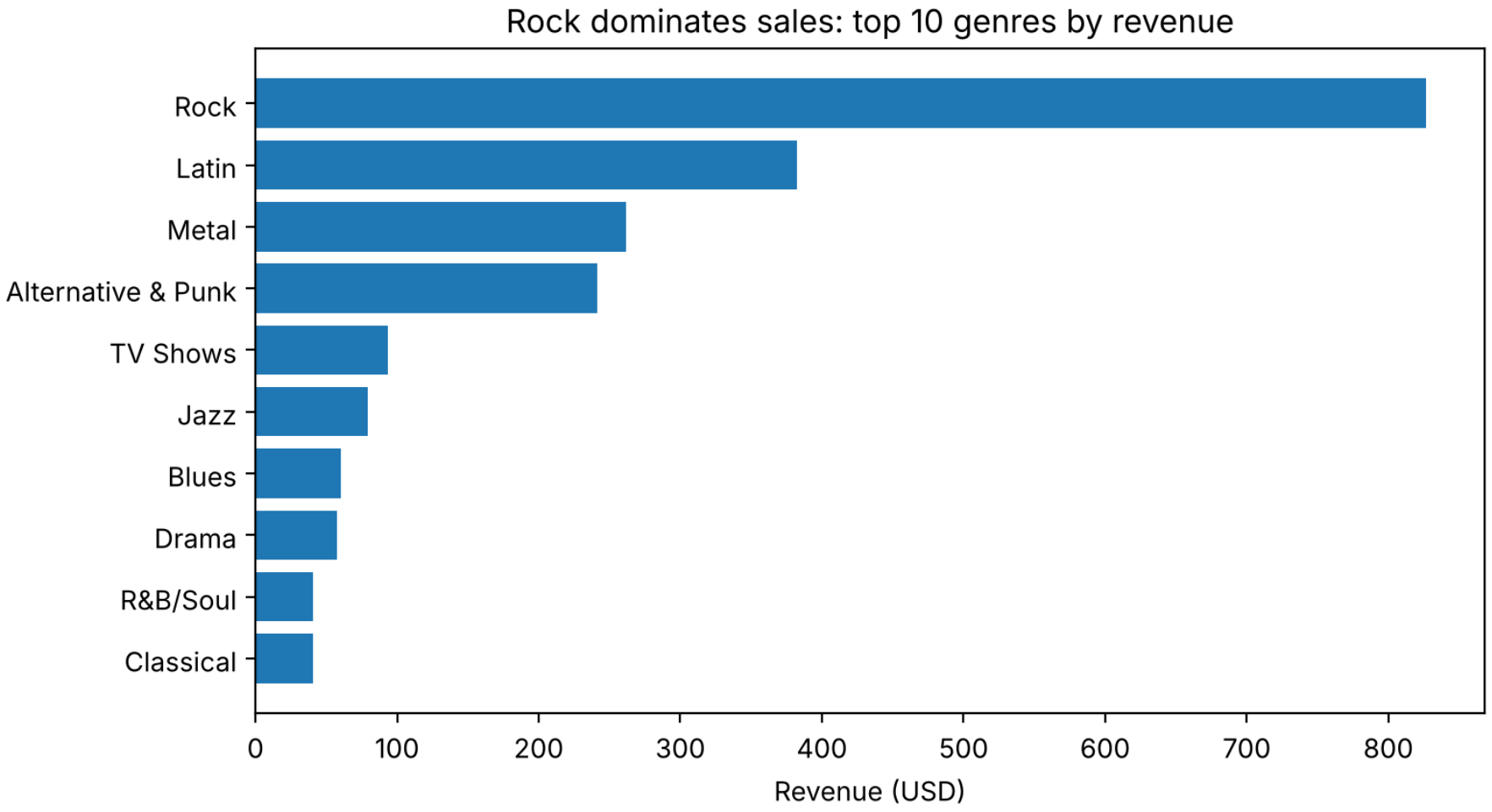
WHERE سطرها را قبل از گروه‌بندی و HAVING گروه‌ها را بعد از آن فیلتر می‌کند.

JOIN: فروش هر سبک موسیقی

- از InvoiceLine (فروش) به Track و از آنجا به Genre
- نام مستعار کوتاه برای جدول‌ها
- LEFT JOIN برای پیدا کردن «آنچه نیست»: ۱۱۰ هنرمند بدون هیچ فروش

```
SELECT g.Name AS genre,
       SUM(il.Quantity) AS tracks_sold,
       ROUND(SUM(il.UnitPrice
                 * il.Quantity), 2) AS revenue
FROM InvoiceLine AS il
JOIN Track AS t ON il.TrackId = t.TrackId
JOIN Genre AS g ON t.GenreId  = g.GenreId
GROUP BY g.Name
ORDER BY revenue DESC
```

نتیجه در Pandas و Matplotlib



WITH (CTE): پرسش‌های چندمرحله‌ای

- هر مرحله یک نام دارد و مثل جدول موقت استفاده می‌شود
- خواناتر از زیرپرسش‌های تودرتو
- مثال: مشتری‌هایی که ۱۰٪ بیشتر از میانگین کشور خودشان خرید کرده‌اند

```
WITH customer_spend AS (
    SELECT c.CustomerId, c.Country,
           SUM(i.Total) AS spent
    FROM Customer c JOIN Invoice i
      ON c.CustomerId = i.CustomerId
    GROUP BY c.CustomerId
), country_avg AS (
    SELECT Country, AVG(spent) AS avg_spent
    FROM customer_spend GROUP BY Country
)
SELECT ... WHERE spent > avg_spent * 1.1
```

توابع پنجره‌ای

- محاسبه روی گروه، اما همه‌ی سطرها حفظ می‌شوند (مثل transform)
- RANK و ROW_NUMBER برای رتبه‌بندی در هر گروه
- LAG و LEAD برای رشد نسبت به دوره‌ی قبل
- SUM() OVER (ORDER BY ...) برای جمع تجمعی

```
SELECT year, revenue,
  100.0 * (revenue - LAG(revenue)
             OVER (ORDER BY year))
        / LAG(revenue) OVER (ORDER BY year)
    AS growth_pct,
  SUM(revenue) OVER (ORDER BY year)
    AS cumulative
FROM yearly

RANK() OVER (PARTITION BY country
             ORDER BY revenue DESC)
```

SQL Injection

```
country = "x' OR '1'='1"

# DANGEROUS: 59 rows leaked
sql(f"SELECT * FROM Customer "
    f"WHERE Country = '{country}'")

# SAFE: 0 rows
sql("SELECT * FROM Customer "
    "WHERE Country = ?",
    params=(country,))
```

- هرگز ورودی کاربر را مستقیم داخل رشته‌ی SQL نچسبانید
- ورودی x' OR '1'='1 شرط را همیشه درست می‌کند و همه‌ی ۵۹ مشتری لو می‌روند
- پرسش پارامتری با ? : مقدار فقط به‌عنوان داده تفسیر می‌شود

SQL و Pandas

SQL	Pandas
SELECT a, b	df[["a","b"]]
WHERE	df[cond] یا query
GROUP BY ... SUM()	groupby().sum()
JOIN	merge
ORDER BY ... LIMIT n	sort_values().head(n)
OVER (PARTITION BY ...)	groupby().transform()

قاعده‌ی سرانگشتی: فیلتر و تجمیع سنگین در پایگاه داده؛ تحلیل آماری و نمودار در Pandas.

جمع‌بندی و تمرین

۱. SELECT، WHERE، GROUP BY، HAVING، JOIN، WITH و توابع پنجره‌ای

۲. همیشه پرسش پارامتری؛ to_sql برای بردن DataFrame به پایگاه داده

۳. تمرین: ۵ آلبوم با بیشترین آهنگ، همراه با نام هنرمند

۴. تمرین: اولین فاکتور هر مشتری با ROW_NUMBER؛ مشتری جدید در هر ماه

۵. تمرین: رشد ماهانه‌ی فروش فروشگاه با LAG

۶. نوت‌بوک: notebooks/07_sql_for_data_analysis.ipynb